Sesión 6

Análisis Avanzado y Transformación de Datos con SQL

Carlos César Sánchez Coronel

BASES DE DATOS

Contenido general

1. Agregaciones y Agrupamientos
2. Funciones de Ventana
3. Limpieza y Transformación de Datos
4. Técnicas Avanzadas para ETL
5. Ejercicios Adicionales
6. Glosario

Contenido de la sección

1. Agregaciones y Agrupamientos
2. Funciones de Ventana
3. Limpieza y Transformación de Datos
4. Técnicas Avanzadas para ETL
5. Ejercicios Adicionales
6. Glosario

Funciones de Agregación Básicas

Funciones estándar

- `COUNT(*)`: número de filas
- `COUNT(expr)`: valores no nulos
- `SUM(expr)`: suma
- `AVG(expr)`: promedio
- `MIN(expr)`: mínimo
- `MAX(expr)`: máximo

GROUP BY y HAVING

Ejemplo: salario promedio por departamento

```sql
SELECT departamento_id,
       COUNT(*) AS num_empleados,
       AVG(salario) AS salario_promedio
FROM empleados
GROUP BY departamento_id;
```

Filtrar grupos con HAVING

```sql
SELECT departamento_id, AVG(salario) AS promedio
FROM empleados
GROUP BY departamento_id
HAVING AVG(salario) > 4000;
```

Agregaciones Avanzadas: ROLLUP, CUBE, GROUPING SETS

ROLLUP – subtotales jerárquicos

```sql
SELECT departamento_id, ciudad, SUM(salario)
FROM empleados
GROUP BY ROLLUP (departamento_id, ciudad);
```

CUBE – todas las combinaciones

```sql
SELECT departamento_id, ciudad, SUM(salario)
FROM empleados
GROUP BY CUBE (departamento_id, ciudad);
```

GROUPING SETS – agrupaciones específicas

```sql
SELECT departamento_id, ciudad, SUM(salario)
FROM empleados
GROUP BY GROUPING SETS ((departamento_id), (ciudad), ());
```

Ejercicio Resuelto: Resumen de Ventas

Enunciado: Tabla `ventas(producto, categoría, monto, fecha)`. Obtener:

1. Monto total por categoría.
2. Monto total por categoría y producto.
3. Categorías con total > 10000.
4. Totales con ROLLUP.

Solución

```sql
-- 1. Por categoría
SELECT categoria, SUM(monto) AS total
FROM ventas GROUP BY categoria;

-- 2. Por categoría y producto
SELECT categoria, producto, SUM(monto)
FROM ventas GROUP BY categoria, producto;

-- 3. Categorías con total > 10000
SELECT categoria, SUM(monto) AS total
FROM ventas
GROUP BY categoria
HAVING SUM(monto) > 10000;

-- 4. Con ROLLUP
SELECT categoria, producto, SUM(monto)
FROM ventas
```

Contenido de la sección

1. Agregaciones y Agrupamientos
2. Funciones de Ventana
3. Limpieza y Transformación de Datos
4. Técnicas Avanzadas para ETL
5. Ejercicios Adicionales
6. Glosario

Concepto y Sintaxis

Definición

- Calculan sobre un conjunto de filas relacionadas con la fila actual.
- No colapsan filas (mismo número de filas de salida).
- Se usan con `OVER()`.

Sintaxis básica

```
funcion() OVER (PARTITION BY col1 ORDER BY col2)
```

Funciones de Ranking

ROW_NUMBER(), RANK(), DENSE_RANK()

```sql
SELECT nombre, salario,
       ROW_NUMBER() OVER (ORDER BY salario DESC) AS row_num,
       RANK()       OVER (ORDER BY salario DESC) AS rank,
       DENSE_RANK() OVER (ORDER BY salario DESC) AS dense_rank
FROM empleados;
```

- ROW_NUMBER: secuencial único
- RANK: mismo rango para empates, deja huecos
- DENSE_RANK: mismo rango, sin huecos

Funciones de Valor: LAG y LEAD

Acceder a filas anteriores/posteriores

```sql
SELECT fecha, monto,
       LAG(monto, 1) OVER (ORDER BY fecha) AS monto_anterior,
       monto - LAG(monto, 1) OVER (ORDER BY fecha) AS diferencia
FROM ventas_diarias;
```

FIRST_VALUE, LAST_VALUE

```sql
SELECT producto, fecha, cantidad,
       FIRST_VALUE(cantidad) OVER (PARTITION BY producto ORDER BY fecha) AS primera_venta
FROM ventas;
```

Agregaciones como Ventana y Marcos

Totales acumulados y medias móviles

```
SELECT fecha, monto,
       SUM(monto) OVER (ORDER BY fecha) AS acumulado,
       AVG(monto) OVER (ORDER BY fecha ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS media_movil_3
FROM ventas_diarias;
```

- `ROWS BETWEEN ... AND ...` define el marco.
- `RANGE` trabaja con valores (ej. fechas).

Ejemplo: Top N por Departamento

Tres empleados mejor pagados por departamento

```
WITH ranked AS (
    SELECT nombre, salario, departamento_id,
           ROW_NUMBER() OVER (PARTITION BY departamento_id ORDER BY salario DESC) AS rn
    FROM empleados
)
SELECT * FROM ranked WHERE rn <= 3;
```

Contenido de la sección

1. Agregaciones y Agrupamientos
2. Funciones de Ventana
3. Limpieza y Transformación de Datos
4. Técnicas Avanzadas para ETL
5. Ejercicios Adicionales
6. Glosario

Manejo de Valores Nulos

COALESCE y NULLIF

```sql
-- Reemplazar nulos
SELECT nombre, COALESCE(email, 'sin-email@empresa.com') AS email
FROM clientes;

-- Evitar división por cero
SELECT total / NULLIF(cantidad, 0) FROM ventas;
```

Funciones de Texto

Manipulación de cadenas

```sql
SELECT UPPER(nombre), LOWER(email), INITCAP(TRIM(nombre))
FROM clientes;

-- Concatenar
SELECT nombre || ' (' || email || ')' FROM clientes;

-- Reemplazar
SELECT REPLACE(telefono, '-', '') FROM clientes;
```

Expresiones Regulares

Extraer dominio de email

```sql
-- PostgreSQL
SELECT email, SUBSTRING(email FROM '@(.*)$') AS dominio
FROM clientes;

-- Con REGEXP_REPLACE
SELECT REGEXP_REPLACE(email, '.*@', '') AS dominio
FROM clientes;
```

Limpiar teléfono (solo dígitos)

```sql
SELECT REGEXP_REPLACE(telefono, '[^0-9]', '', 'g') AS telefono_limpio
FROM clientes;
```

Funciones de Fecha y Hora

Extraer partes, truncar, convertir

```sql
-- Extraer año
SELECT EXTRACT(YEAR FROM fecha) FROM ventas;

-- Truncar a mes
SELECT DATE_TRUNC('month', fecha) FROM ventas;

-- Convertir texto a fecha
SELECT TO_DATE('15/03/2025', 'DD/MM/YYYY') AS fecha;
```

Ejercicio Resuelto: Limpieza de Datos de Clientes

Enunciado: Tabla `clientes_raw` con columnas: `id`, `nombre` (espacios y mayúsculas inconsistentes), `email` (nulos), `fecha_registro` (texto 'DD/MM/YYYY'). Limpiar:

1. Nombre en formato título, sin espacios extras.
2. Email: si es nulo, asignar 'pendiente@mail.com'.
3. Fecha convertida a tipo DATE.

Solución

```sql
SELECT id,
       INITCAP(TRIM(nombre)) AS nombre_limpio,
       COALESCE(email, 'pendiente@mail.com') AS email,
       TO_DATE(fecha_registro, 'DD/MM/YYYY') AS fecha_registro_date
FROM clientes_raw;
```

Contenido de la sección

1. Agregaciones y Agrupamientos
2. Funciones de Ventana
3. Limpieza y Transformación de Datos
4. Técnicas Avanzadas para ETL
5. Ejercicios Adicionales
6. Glosario

Tablas Temporales

Crear y usar tablas temporales

```sql
CREATE TEMP TABLE ventas_trimestre AS
SELECT *
FROM ventas
WHERE fecha >= DATE_TRUNC('quarter', CURRENT_DATE) - INTERVAL '3 months';

SELECT * FROM ventas_trimestre;
```

- Existen solo durante la sesión.
- Útiles para resultados intermedios.

CTEs y Subconsultas Correlacionadas

CTE para mejor legibilidad

```sql
WITH promedio_depto AS (
    SELECT departamento_id, AVG(salario) AS salario_prom
    FROM empleados GROUP BY departamento_id
)
SELECT e.nombre, e.salario, pd.salario_prom
FROM empleados e
JOIN promedio_depto pd ON e.departamento_id = pd.departamento_id
WHERE e.salario > pd.salario_prom;
```

Condicionales con CASE

Categorizar salarios

```
SELECT nombre, salario,
       CASE
           WHEN salario < 2000 THEN 'Bajo'
           WHEN salario < 4000 THEN 'Medio'
           ELSE 'Alto'
       END AS categoria_salarial
FROM empleados;
```

Pivotes con CASE y Agregación Condicional

Ventas mensuales por producto

```
SELECT producto,
       SUM(CASE WHEN EXTRACT(month FROM fecha) = 1 THEN monto ELSE 0 END) AS enero,
       SUM(CASE WHEN EXTRACT(month FROM fecha) = 2 THEN monto ELSE 0 END) AS febrero,
       SUM(monto) AS total
FROM ventas
GROUP BY producto;
```

Usando FILTER (PostgreSQL)

```
SELECT producto,
       SUM(monto) FILTER (WHERE EXTRACT(month FROM fecha) = 1) AS enero,
       SUM(monto) FILTER (WHERE EXTRACT(month FROM fecha) = 2) AS febrero
FROM ventas
GROUP BY producto;
```

Manejo de Duplicados con ROW_NUMBER

Eliminar duplicados por email

```sql
WITH duplicados AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY email ORDER BY id) AS rn
    FROM clientes
)
DELETE FROM clientes
WHERE id IN (SELECT id FROM duplicados WHERE rn > 1);
```

Ejercicio Resuelto: Transformación Compleja

Enunciado: Con tablas ventas y productos, se pide:

1. Crear tabla temporal con ventas del último trimestre.
2. Pivot: total por categoría y mes.
3. Clasificar ventas como 'baja', 'media', 'alta'.
4. Mostrar solo categorías con total > 1000.

Solución (parcial)

```sql
-- 1. Tabla temporal
CREATE TEMP TABLE ventas_trimestre AS
SELECT v.*, p.categoria
FROM ventas v
JOIN productos p ON v.producto = p.id
WHERE v.fecha >= DATE_TRUNC('quarter', CURRENT_DATE) - INTERVAL '3 months';

-- 2. Pivot por mes
SELECT categoria,
       SUM(CASE WHEN EXTRACT(month FROM fecha) = 1 THEN monto ELSE 0 END) AS enero,
       SUM(CASE WHEN EXTRACT(month FROM fecha) = 2 THEN monto ELSE 0 END) AS febrero,
       SUM(monto) AS total
FROM ventas_trimestre
GROUP BY categoria
HAVING SUM(monto) > 1000;
```

Contenido de la sección

1. Agregaciones y Agrupamientos
2. Funciones de Ventana
3. Limpieza y Transformación de Datos
4. Técnicas Avanzadas para ETL
5. Ejercicios Adicionales
6. Glosario

Ejercicio 1: Series Temporales

Enunciado: Calcular la media móvil de 7 días de las ventas diarias.

Solución

```sql
SELECT fecha, monto,
       AVG(monto) OVER (ORDER BY fecha ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS media_movil_7
FROM ventas_diarias;
```

Ejercicio 2: Extraer Código Postal

Enunciado: De una dirección como 'Calle Falsa 123, 28080 Madrid', extraer el código postal (5 dígitos).

Solución

```sql
SELECT direccion,
       SUBSTRING(direccion FROM '[0-9]{5}') AS codigo_postal
FROM clientes;
```

Ejercicio 3: Agregación Condicional

Enunciado: Contar empleados por departamento y cuántos tienen salario > 3000.

Solución

```sql
SELECT departamento_id,
       COUNT(*) AS total,
       COUNT(*) FILTER (WHERE salario > 3000) AS altos_salarios
FROM empleados
GROUP BY departamento_id;
```

Ejercicio 4: CTE Recursiva para Jerarquía

Enunciado: Con tabla `empleados(id, nombre, jefe_id)`, mostrar nivel jerárquico y camino desde el CEO.

Solución

```sql
WITH RECURSIVE jerarquia AS (
    SELECT id, nombre, jefe_id, 1 AS nivel, nombre AS path
    FROM empleados WHERE jefe_id IS NULL
    UNION ALL
    SELECT e.id, e.nombre, e.jefe_id, j.nivel + 1,
           j.path || ' > ' || e.nombre
    FROM empleados e
    JOIN jerarquia j ON e.jefe_id = j.id
)
SELECT id, nombre, nivel, path
FROM jerarquia
ORDER BY nivel, nombre;
```

Contenido de la sección

1. Agregaciones y Agrupamientos
2. Funciones de Ventana
3. Limpieza y Transformación de Datos
4. Técnicas Avanzadas para ETL
5. Ejercicios Adicionales
6. Glosario

Términos Clave

Función de agregación Opera sobre un conjunto y devuelve un único valor.

GROUP BY Agrupa filas con valores comunes.

HAVING Filtra grupos después de la agregación.

ROLLUP / CUBE / GROUPING SETS Extensiones para subtotales.

Función de ventana Calcula sobre un marco de filas sin colapsar.

OVER Define la ventana (partición, orden, marco).

ROW_NUMBER, RANK, DENSE_RANK Funciones de ranking.

LAG / LEAD Acceden a filas anteriores/posteriores.

COALESCE Primer valor no nulo.

NULLIF Compara y devuelve NULL si iguales.

Expresión regular Patrón para búsqueda en texto.

CTE Common Table Expression (WITH).

Tabla temporal Tabla efímera para una sesión.

CASE Expresión condicional.

Pivot Transformar filas en columnas.

¡Gracias por su atención!